Inference without trained weights

Ikailo John Sessford
Ikailo Inc., Whitby, Ontario, Canada
john@ikailo.com ORCID 0009-0002-0150-4027

Pre-print. November 2026.

Abstract

A transformer computes its attention pattern at read time, from the data. This paper computes the read's other inputs the same way: a vertex's coordinate and the weight a relation carries are derived from a store of statements at the moment a question is asked, in one basis that both sides of a relation share, with no fitted constant. What results infers: it names facts the store does not contain, with the fact being asked about removed from every part of the material the answer is read from.

A vertex's coordinate is the set of things it is connected to, recomputed from the statements and stored nowhere. A relation's weight is the cross-covariance of the statements that use it. Attention is an alignment standardized by the exact permutation null, with no temperature and no forced normalization. A fact enters by one write, and the next read has it.

On a hand-written store of 84 statements the read names 36.8% of 68 held-out facts from a pool averaging 64.7 candidates, 23.8 times a draw over the same candidates. It survives a lookup control and clears a degree-preserving permutation null on two of three WordNet neighbourhoods, and the third does not clear it. A TransE model retrained for every held-out question and tuned over dimension and margin does better at its best, at 53.4%.

On generated records with an ordered axis the score recovers a broad true key in every cell of a sweep where a dot product fails and a cosine degrades, is right wherever it answers at least five times, and counts relations up to the limit the basis sets. That advantage does not reproduce on any real corpus measured. On five real corpora the score is competitive at best and never ahead of the best scalar, and in the questions of one corpus where the arithmetic most favours it, it has no skill above chance. On text it does not beat counting, and on next-word prediction an n-gram lookup wins and the instrument's own reads do not beat a dot product. The algebra the read rests on is machine-checked in Lean 4 against Mathlib.

1. What the read uses, item by item

a trained model holds	the derived read uses
an embedding table	a place: each vertex's coordinate is what it is connected to, recomputed from the statements at read time and stored nowhere
W_Q, W_K, W_V	nothing; both sides of a relation already sit in one basis, so there is no projection to learn
a relation's weight matrix	the cross-covariance of the statements that use the relation, computed in one pass
$\mathrm{softmax}(q \cdot k/\sqrt{d})$	an alignment standardized by the exact permutation null, with no temperature and no forced normalization
a fact learned by retraining	one write; the next read derives a different operator because the statement is in the sum

The last row separates the two kinds of system. A fact learned after training enters a trained model's weights only by retraining it. Here a fact enters by being written to the store, and the next read has it, because the read derives the operator from whatever the store holds at the moment it is asked.

One number is not derived: the false-alarm level α. By Neyman–Pearson [1] it belongs to the reader, so every reading that decides something names the level it is taken at, and the attention layer of §4.1 is swept across three levels. The reads of §4.2 to §4.9 rank and name a candidate and take no level.

2. The method

2.1 The store

A store is a set of statements. A statement is a tuple of vertices of any width; the usual case is a triple (a, r, b). Any vertex that stands strictly inside a statement, with something before it and something after it, is used as a relation there. Relations are counted off the store, so no list of relation names exists anywhere, and the same vertex can be a relation in one statement and an end in another. Each statement is itself a vertex, joined to each of its parts.

2.2 The basis

A coordinate lives on axes counted off the store. Each part of each statement contributes one axis: its position paired with the rest of the statement, $(p, s \setminus s_p)$. Each statement adds one axis of its own. Binding the position to the rest of the statement keeps which relation went with which target; an axis per position and vertex alone would sum those together and lose the pairing. The basis grows when a statement arrives, and no size is chosen.

The edges run both ways. A part s_p of statement s is joined to s, arriving on the axis $(p, s \setminus s_p)$; the statement is joined to each of its parts, arriving on its own axis.

2.3 The walk and the place

A vertex's place is built by a walk outward from it. The start carries the value 1. A vertex reached at value v with

d neighbours offers each of them v/d, so a statement with three parts divides by three and a vertex used in twenty statements divides by twenty, and a hub does not flood everything hanging off it. The decay is the store's own branching.

Each vertex is settled once, at the largest value that reaches it, largest first. This is a max-product shortest path, so the place cannot depend on the order the statements were written down in. The place is the sum, over the settled vertices, of each vertex's value added into every axis it points at.

The walk stops at its own aperture. Read the settled values as a power spectrum, $w_i = v_i^2$, and take its entropy $H = \log_2 S - L/S$, with $S = \sum w_i$ and $L = \sum w_i \log_2 w_i$. The participation number 2^H is the effective count of occupied cells. Because the walk settles largest first, every value still to come is at most the last one settled, and that bounds the unsettled tail. The walk takes 2^H twice, once with the tail at nothing and once with the tail at its largest possible spread, and stops when both round to the same count. The place keeps every vertex at or above the value of the last one that count admits. Nothing about where the walk stops is supplied by anyone.

Places are recomputed on every read. A coordinate that persisted across a write would be a trained object, and a fact added to the store would reach it only through a rebuild.

2.4 The operator

For a relation r, take every statement that uses it. Stack the places of what stands immediately before r as the rows of A, and the places of what stands immediately after it as the rows of B, so that row i of both belongs to the same statement. Centre the columns of each, giving $\tilde{A}$ and $\tilde{B}$. The operator is the cross-covariance

$$M_r = \tilde{A}^\top \tilde{B}.$$

It is a statistic of the statements: there is no target, no residual, no learning rate and nothing iterated. A statement arriving adds one row to each side, which is one rank-one term in the sum. A relation used by no surviving statement has no operator, and neither has a relation used exactly once, because a cross-covariance over one row is that row and it predicts that row and nothing else.

2.5 The landing

To ask where r takes a source vertex with place u, centre the source, $\tilde{u} = u - \bar{u}$, and put it through the operator:

$$\ell = \tilde{u} M_r = (\tilde{u}\tilde{A}^\top)\tilde{B}.$$

The right-hand side never forms M_r. It costs time linear in the number of statements using the relation, where forming the matrix costs the square of the width of the basis, and that is what makes a wide store readable at all.

2.6 The score and the attention row

Each candidate is scored by the cosine between its place and the landing. The candidate with the largest positive score is named; a row with no positive score names nothing. Nothing forces the row to sum to anything, so a row of candidates that are all wrong is a row of small numbers, and the reader sees it as one.

In the attention layer the same alignment between a query frame and a key frame is standardized by the exact permutation null of re-pairing their rows. The null's mean and second moment are in closed form, the permutation moments of Pitman [2] and Hoeffding [3], so there is no sampling and no random generator. An alignment below the cutoff the false-alarm level implies is set to zero, which makes refusal arithmetic: nothing above the level means nothing out.

2.7 Learning

Learning is a write. A new statement enters the basis, the walk and the operator of every relation it uses at the next read, as one rank-one term. There is no update rule, no rate and no pass over a training set.

3. The protocol

3.1 Holding the answer out

When the read is asked what relation r reaches from a, the statement (a, r, b) is removed from the store before anything is computed, out of the operator and out of the coordinate. Removing it from the operator alone leaves it in place: a's coordinate is built from the edges it carries, so a fact left in the coordinate is still stated in the material the answer is read from. The cost of that mistake is measured on one step of the reader, told which relation acts and choosing among a candidate set of about 1 in 31. Built once from the whole store, the bound coordinate names 60.5% of the 81 held-out facts it is asked; rebuilt without each one, it names 27.9% of the 68 a rebuilt store can still pose. The two are different question sets, so the drop is not a paired figure, and every structure measured falls by more than half.

This is a different read from the 36.8% of §4.2. That figure is the bare operator, with no step of the reader around it, ranking a pool averaging 64.7 candidates. Both are taken with the statement held out of operator and coordinate alike; they differ in the read and in the candidate set.

Every answer reports whether the store, as read, still contains the statement asked about, beside the size of the pool and the number of candidates the relation reaches elsewhere in the store.

3.2 The bar to beat

Naming one candidate of 65 is not the achievement when the relation only ever points at 12 things. So every result reports two baselines: a draw over the pool the read saw, and a draw among the candidates the relation already

reaches. The second is the figure that knowing which relation was asked cannot win.

3.3 The four ways a held-out answer still gets stated

Holding the statement out of the coordinate is necessary and it is not sufficient. A store can state the same fact more than once, and each of these is a leak in a protocol that looks sound.

the leak	what it looks like	how it is closed
left in the coordinate	the operator is rebuilt but the basis is not	on the reader step of §3.1, the bound coordinate reads 60.5% of 81 built once against 27.9% of 68 rebuilt; every structure falls by more than half
stored backwards under an inverse label	$(a, \text{hypernym}, b)$ and $(b, \text{hyponym}, a)$	one direction of each pair is kept
stored backwards under the same label, because the label is its own inverse	$(a, \text{attribute}, b)$ and $(b, \text{attribute}, a)$	the reciprocity check drops the label; WordNet's attribute and also-see relations go this way
stored again under a different label	(a, aligns, b) and $(a, \text{colimit_of}, b)$	the hold-out removes every label joining the pair

The third defeats the standard guard against the second. Keeping one direction of each inverse pair is the usual fix, and a symmetric relation has no second name to leave out, so that rule never reaches it.

The fourth hides. Neither label is the other's inverse, nothing in their names suggests overlap, and a reciprocity check passes it. The tell is two labels with exactly equal statement counts.

Both are counted. Reciprocity is measured per label and the number of labels joining each pair of vertices is measured per pair, on every store read, before any result is taken. So the hold-out removes every statement joining the two vertices, under any label, because the question is about the pair; and any relation with a reverse anywhere in the store is dropped before a single reading is taken.

3.4 The controls

Two further runs stand beside every claim on the same store and the same questions.

The lookup arm leaves the asked-about statement in the coordinate and the operator. It must score higher, and the gap between the two is what the hold-out costs.

The permutation null shuffles the destinations inside each relation. Every concept keeps its in-degree and out-degree, every relation keeps its size, the pools are identical, and the only thing removed is which source reaches which destination. A claim is accepted when it clears its own null, not when it beats a draw. With d permuted stores the smallest attainable share is $1/(d+1)$; twenty draws give a floor of 0.048. The null holds a condition, and the run checks it: shuffling inside each relation is sound only where a pair of vertices carries one relation. Where a pair carries two, the shuffle dissolves the doubling and the permuted store has more distinct edges than the real one, which is a different graph, so the run refuses to produce a figure there.

4. What is measured

4.1 The layer, counted against standard attention

On frames generated from distinct random bases, with the query built from one of them plus noise, the derived attention layer is set beside a standard multi-head attention layer [4] of the same width:

layer	tensors	numbers to train
derived attention	0	0
multi-head attention, same width	4	1088

The derived layer names the right key on every draw at each of the three false-alarm levels out to noise amplitude 4, where a draw among 6 keys is right one time in six, having never been shown which key anything came from. At $\alpha = 0.001$ the mean row sum runs from 0.988 at noise 0.25 down to 0.398 at noise 4; at noise 64 it goes slightly negative at $\alpha = 0.05$ and 0.01. A softmax puts every row at exactly 1.000 by construction.

The sweep runs until it fails, because a result with no located edge does not tell a reader where the method stops. Past noise 8 the frames are buried, and how it fails is the finding:

α	answered at noise 64	right at noise 64
0.05	25%	0%
0.01	10%	5%
0.001	0%	0%

Tightening the level turns a wrong answer into a refusal. That is what a false-alarm level is for. A softmax returns a distribution over the keys whatever is in them and has no way to report that there was nothing there. The frames here are records with an ordered axis, and §5 measures the score on such records directly.

4.2 The bare operator on a hand-written store

The store is a hand-written ontology of 84 statements, written before any of these questions were framed. The read is the association alone: no floor, no ordering rule and no constraint, with the statement held out of both operator and coordinate.

- 36.8% of 68 held-out facts are named, from a pool averaging 64.7 candidates.
- A draw over the same candidates gives 1.5%, so the read is 23.8 times the draw, at p = 7.51e-28.
- The truth sits 0.28 of the way down the operator's own ordering, where a draw leaves it at the middle.

- Among the 59 questions whose answer the relation points into elsewhere, the read names 40.7% against 11.2% for a draw among those candidates, 3.6 times.

The last figure is the one that knowing which relation was asked cannot win.

4.3 The ceiling a generated store imposes

The same measurement on a generated store gives 7.9 to 8.5 times the draw overall, and 0.97 to 1.06 times once inside the group the relation reaches, which is a draw.

That generator draws each destination uniformly from the destination group, so which concept in the group a statement names is decided by nothing, and an operator at a draw there reads everything the store contains. That is why the store-level figures of §4.2 and §4.4 to §4.9 are taken on the hand-written store, on WordNet and on prose. The generated frames of §4.1 and the generated stores of this section bound what the read can do on a structure whose answer is known; they are not offered as its performance on a store.

4.4 One implementation

The research measurements and the shipped engine are one implementation, and the identity is measured on the hand-written store:

compared	largest difference
every coordinate	0.000e+00
every operator	0.000e+00
every landing	0.000e+00

All three are zero by construction and are measured anyway, because an argument that two paths are one goes stale when somebody adds a copy back. The refusals agree too: where a relation is used once, both paths decline. The landing is computed with the operator never formed (§2.5).

The same store is written into a Mantle lattice, 84 statements as vertices and 252 edges carrying only the position each part arrives at, read back, and asked the same questions. The store survives exactly and every probe names what it named in memory. No relation kind is written and none is read: a relation is a vertex that happens to be used in the middle of statements.

4.5 A relation arriving, one statement at a time

For each held-out question the store starts with the relation being asked about entirely absent, so that no statement mentions it, and its other statements are put back one at a time, the held-out one never.

15 of 26 questions are never right within 24 additions, and the part-of relation gets none of its six right at any count. A relation arriving does not reliably make a question about it answerable, and on this store it more often does not.

Where it does arrive, it arrives early: first answer at a mean of 3.0 statements, and first correct at a mean of 2.7 to 4.7 statements across the three relations that get any question right, out of relations carrying 2 to 35. What a relation needs is a handful of uses, and where the answer never comes, more of the same relation does not bring it. No relation answers from a single use, which is the guard of §2.4 at work.

A statement arrives in 4.3 ms: the write, the coordinate rebuilt, the operator rederived and the question answered, the median on the measuring machine. There is no gradient step, no schedule and no pass over a training set.

4.6 The two runs that would explain the result away

The controls of §3.4 run on three WordNet neighbourhoods, each a connected region walked out from a seed synset.

The lookup arm scores 100.0%, 100.0% and 98.9% on the entity.n.01, dog.n.01 and vehicle.n.01 neighbourhoods, against the read's 20.7%, 19.7% and 6.0%. The protocol takes each to between about a sixteenth and about a fifth of its lookup score.

The permutation null, twenty stores per neighbourhood, reads among the reached candidates:

A neighbourhood clears its null when the share of permuted stores at or above its claim is at most $\alpha = 0.05$. With twenty stores the smallest share there is 0.048, reached only when no permuted store matches the claim. Two neighbourhoods clear at that level. vehicle.n.01 does not: one of twenty stores with no correspondence in them reaches its figure, its share of 0.095 is above the level, and its null worst sits above its claim. It beats a draw among the reached at 2.9 times while a store with the correspondence removed reaches the same figure, which is why beating a draw is not acceptance and clearing the permuted null is.

The permutation is conservative. Where a relation sends many sources to one destination it re-creates some true pairs by arithmetic, so it holds more structure than a clean shuffle, and a claim has to clear it anyway. The doubling condition of §3.4 is counted per neighbourhood and is zero on all three.

4.7 Text, at the one scale measured so far

Five task runs put the read on text. On four of them, which read text as co-occurrence statistics, the read wins none: on a held-out collocate task counting beats the association on Moby-Dick and the two are not separated on Alice; on the next word from a multi-word context counting wins in four of ten cells and the rest are not separated; on a second-order task a cosine between context vectors beats the subspace read on both books. The fifth reads text as recorded spans.

Prose has to be cut into statements before the coordinate exists. An axis is a position plus the rest of the statement, so two vertices meet on one only where the remainder repeats. The ingestion has no width argument: it cuts each position at the extent its own middle earns,

seed	the claim	null median	null worst	draws at or above the claim	share
entity.n.01	5.0	1.2	2.5	0 of 20	0.048
vehicle.n.01	2.9	1.7	3.3	1 of 20	0.095
dog.n.01	5.0	1.9	4.9	0 of 20	0.048

which on a body of 120 sentences writes 1250 spans of two tokens, 1284 of three and 38 of four. Over 400 sentences, on the read's own basis:

store	share of a vertex's axes shared with another vertex
the hand-written triples	54.7%
prose, as the ingestion writes it	63.7%

The prose store shares its coordinate at least as much as the triples do, and a span has the same shape as a triple.

The read does not beat counting on it. On 114 held-out spans out of 2572, counting names the next token 7.0% of the time and so does the read, paired 6 to 6 with 2 both right, where the smallest p that pairing could produce is 0.0005. The hold-out removes every span joining the two tokens, because a trigram store records the same pair of tokens under many middles and one left behind names the same next token.

On the larger stores below, counting wins, and the ordering measurement says by how much. Both families of negative infer the relation from position: the co-occurrence runs from counts, this one from adjacency. Adjacency records order, and the criterion this program states, an asymmetric relation that is recorded rather than inferred, is not met by putting three tokens next to each other.

A top-1 share cannot say how a read loses. Scored as an ordering instead, with the counter ranked by its own counts, on 1601 held-out questions over 1612 candidates each:

An ordering carrying nothing puts the answer halfway down the pool, at 806.5. The read is not narrowly beaten; the answer is far from the top, and no sharpening of the last discrimination reaches a median of 218.

One mechanism for that is tested and refused. The relation here is the span's middle token, so the operator for a common word averages over every pair that ever had it between them, which should point nowhere. Bucketing the same questions by how many statements the relation has collected gives median ranks of 241, 257, 251, 1256, 155, 179 and 124.5, from relations holding one or two statements to those holding a thousand or more. There is no monotone trend, and the most crowded relations hold the best median of all, so crowding does not make the read worse. What costs the read is open.

A fitted model does no better on the same questions. TransE [5], swept over dimension and margin under several seeds and trained on the same training store, is put on prose cut into three-token spans over six public-domain books, with whole sentences held out before anything is built. On the 961 held-out spans that survive there, counting names 16.2%, the derived read 12.8%, 0.79 times the counter, and TransE at its best setting 3.1%, 0.19 times it. The fitted model loses to counting too, so what separates both arms from the counter is how adjacency records a corpus, and the negative on text is a fact about that representation. On the two stores where an asymmetric relation is recorded, both arms beat a draw by a wide margin and the fitted one wins.

A span is one rung of a ladder. A token is a vertex; the text it sits in is also a vertex; so is the text that sits in. Written as $(\text{token}, \text{placed_in_}X, X)$, a statement like any other with the membership token earning its operator from the statements that use it, the same prose carries four scales, each measured for what it buys and what it costs:

store	askable under hold-out	steps for a read through the median relation
2572 spans	97.7%	3186
plus its 120 sentences	98.8%	2631
plus its 39 paragraphs	99.1%	2751
plus the document	99.2%	2898

Every rung buys something, and each buys less than the one below it. A rung's gain in askable share, divided by the ratio of its read cost to the cost of the rung below, runs 1.29, 0.33 and 0.08 points. The sentence rung buys +1.1% and makes the read through the median relation cheaper, 0.83 times the read below it. The document is the limit of the model and behaves like it: one relation naming every token, relating everything to everything, buying +0.08% for 1.05 times the read. A scale is worth climbing while it groups tokens that belong together, and stops paying when it groups all of them.

Two thresholds are reported because the protocol demands it. A relation needs 2 uses to have an operator; it needs 3 to still have one when the statement asked about is removed. Measuring at the first overstates every point: the spans read 99.9% askable in the store and 97.7% under the protocol every figure here is taken with.

Askable is not answered. Asked of the sentence rung with held-out questions, on 114 held-out spans the read names 7.0% on the spans alone, 1.8% with the sentences added, and 4.4% on a store whose sentences are permuted. The sentence rung makes more of the store askable and the read worse, below its own permutation null.

	top 1	top 5	top 10	top 50	median rank	mean 1/rank
the read	10.9%	20.2%	24.7%	35.0%	218	0.157
counting	17.0%	36.4%	45.0%	64.8%	15	0.263

4.8 What a read costs, and how it grows

The coordinate is the whole cost of a read. Read to its end, the walk of §2.3 reaches the whole connected region: a place is non-zero on 100% of the axes at every store size from 50 to 705 statements, and the non-zero entries grow 185.9 times over a store that grows 14.1 times, an exponent of 1.975, where linear is 1 and quadratic is 2.

The walk stops at its own aperture instead. A place then touches 14.1% of the axes at 50 statements and 10.4% at 705, an exponent of 1.860. The read keeps or gains on two of three WordNet stores and gives a little on the third: 25.0% top-1 and 4.5 times the draw among the reached on 48 questions either way; 20.7% against 22.4% read to its end on 58, at 5.0 times either way; and 26.2% and 19.0 times against 18.6% and 13.2 on 172. On the hand-written store a place resolves 4.12 vertices of 150 and names 36.8% of 68 held-out statements, against 32.4% read to its end. Every figure in this paper is taken on the walk that stops at its aperture.

The cost divides into two regimes. At 363 statements, on the measuring machine, a held-out question costs 0.0343 s, the coordinate 0.0309 s, building the operator 0.0153 s and applying it 0.0014 s, each the median of its own repeated calls. The coordinate is rebuilt per question only because the protocol removes the asked-about statement from it. Asked something the store does not already hold, there is nothing to hold out, and the question costs 0.0014 s. The seconds belong to that machine; the proportions are the finding. Reading is cheap, and measuring it soundly is what costs.

4.9 Against a model that is trained, on the same store

The sections above show the read working with nothing fitted. How it compares to something fitted is a separate statement, and this section measures it.

TransE, the canonical model for naming the missing end of a triple, is retrained from scratch for every held-out question, so it never sees the answer. Its dimension and margin are swept over nine settings and its best is compared, against a read that has nothing to sweep. Its epochs (300) and learning rate (0.05) are fixed, which bounds the comparison: a TransE tuned over those as well can score higher still.

on the hand-written store, 68 held-out questions	overall	among the reached (59)
the derived read	36.8%	40.7%
TransE, best mean (dimension 16, margin 2.0)	53.4%	61.6%
TransE, weakest overall (dimension 16, margin 0.5)	32.8%	50.8%
TransE, weakest among the reached (dimension 64, margin 0.5)	34.8%	45.8%

The trained model wins at its best, and among the reached it wins at all nine settings; its weakest there is 45.8% against 40.7%. Over the whole pool it beats the derived read at six of the nine settings, ties it at one (dimension 32, margin 0.5, 36.8%) and falls below it at two. Paired at its best it is right alone on 14.0 questions and the derived read alone on 2.67, means over three seeds.

The seeds are swept and the spread is reported because the best setting depends on the draw. At its best setting TransE runs from 45.6% to 61.8% across three seeds.

So the claim this paper supports is exactly this: the read infers, with no fitted constant anywhere, on material the statement has been removed from, and it does so less well than a fitted model given the same material and tuned. Every statement in this paper about what the read does is read against this table.

The spread is the trained model's. Over nine settings its mean runs from 32.8% to 53.4%, and at its weakest it is below the derived read on the whole pool and still above it among the reached. The derived read is one number because there is nothing to tune.

This is one store of 84 statements. On the prose stores of §4.7, TransE at its best names 3.1% against the derived read's 12.8%, and counting beats both. §5.5 puts a second trained model on generated records, and §8 names what varies from here.

5. The score on records with an ordered axis

The attention layer of §4.1 reads records that carry an ordered axis: each is a field of T rows and D channels, and two records meet on one basis. This section measures the score on such records, generated fresh from a stated process at a fixed seed, so that the right answer is known for every question. The score is the coupling read of the Entroptics instrument [6]: the alignment $S = \langle \tilde{A}, \tilde{B} \rangle_F$ of the two centred records, standardized by the exact null of re-pairing their rows, whose variance is $\mathrm{tr}(C_a C_b)/(T-1)$ with $C_a = \tilde{A}^\top \tilde{A}$ and $C_b = \tilde{B}^\top \tilde{B}$. A cosine divides by

$\|\tilde{A}\|\|\tilde{B}\|$. The two agree only where each record's covariance is isotropic, which is where a record has no internal structure, and everywhere else they differ. §9 proves the null's mean and second moment and checks that the code divides by the proved quantity. Every read in this section that decides something is taken at $\alpha = 0.05$, and so is every such read in §6.1 to §6.5.

5.1 Selection, and where it degrades

A broad true key shares every facet of the query, moderately. Narrow distractors share fewer facets, louder. A distractor carrying m of R facets at gain g shares $g\sqrt{m/R}$ of the query's energy against the true key's 1, so above $g = \sqrt{R/m}$ it carries more shared energy and any method ranking by shared magnitude must fail. The crossover is arithmetic and is known before the run. Every record carries the same total signal energy, so a facet count and a signal-to-noise ratio cannot be confounded. With $T = 96$, $D = 32$, 12 candidates per query and 80 trials per cell, top-1 accuracy at $g = 3$:

The score is 1.000 in all 30 cells of the facet and gain sweep. The dot product collapses to 0.000 past the crossover, as the arithmetic requires. Collapsing the ordered axis, the analogue of standard attention, fails from two facets up. At one facet the cosine and the dot product tie with the score, which is the control: one facet is one scalar's worth of evidence, and a read that won there would be reading something other than facet structure.

The mechanism is the gap between the cosine and the score. A narrow loud distractor has a concentrated covariance, which inflates $\mathrm{tr}(C_a C_b)$ and deflates its standardized alignment; a broad key whose covariance matches the query's does not. A product of magnitudes cannot see that difference.

That table holds the width at 32 channels and the noise at 0.10. Both are swept underneath it, over 575 cells: four widths, five noise levels, six facet counts and five gains, at 40 trials per cell.

mean of the score	11 channels	18	32	64
noise 0.10 and below	1.000	1.000	1.000	1.000
noise 0.20	0.901	0.872	0.776	0.653
noise 0.35	0.459	0.470	0.418	0.295

The crossover holds everywhere: past it the dot product is at or below chance in 260 of 260 cells. The score is 1.000 at every width for every noise level at or below the table's 0.10, and 1.000 in 435 of the 575 cells. It degrades with the noise, to between 0.653 and 0.901 at 0.20 and between 0.295 and 0.470 at 0.35, so the held width is inert for this claim and the held noise buys the level. A read that falls as its records get noisier is behaving as a read should. A scalar beats the score in 36 of the 575 cells, the worst by 0.15. A further 25 cells cannot be built at all, because the one-facet control needs one disjoint facet per key.

5.2 Refusal: precision, coverage and what it costs

An alignment always has a maximum, so a read that returns its argmax always answers. A second, independent read gates it. The alignment proposes the candidate with the strongest standardized score among those that resolve. The proposal is returned only if it is also the unique candidate sharing the most resolved channels with the query, each count a resolved rank above the instrument's own floor; where two tie, nothing is returned. The two reads agree either by both being right or by coincidence, and requiring uniqueness removes the coincidence. No threshold enters. Breadth catches what alignment cannot: a narrow, loud, consistent partial match is the same wrong answer on every subsample of a record, so resampling confirms it, but it cannot share the query's channels that it does not carry.

The breadth ratio and the noise are swept rather than drawn, at 80 trials per cell:

Precision is 1.000 in every cell where the decision answers at least five times. Where alignment alone is at chance it answers on none of 80 trials in seven of the ten cells, and the other three answer once, three times and four times, wrong every time. The cut of five answers is therefore load-bearing: three cells are unsound at a cut of one, two at two and three, one at four, and none from five to 20. The cost is coverage. In twenty-eight of the thirty-six cells the decision refuses answers that unguarded alignment would have got right, and in the worst it refuses every one of them. A refusal is reported with its reason, since nothing resolving, nothing sharing a channel and two candidates indistinguishable are different facts about the records.

5.3 The forward pass, and its controls

Inference here is two of a triple in and the third out. The operator cannot be named, because there is no vocabulary to name it from, so it is supplied by example: given a pair (c, d) standing in some relation and a source a, which candidate b stands in that relation to a? The records are built so that the operator has to do the work. Every source is related to every one of its candidates, each by a different kind, so a read that ignores the operator cannot beat chance among them. With $T = 96$, $D = 32$, 6 sources, noise 0.10 and 12 draws:

Ranking candidates by raw association strength, the same read with the operator taken out, sits at chance to the digit at every R, so every point above it is the operator's. A wrong-kind exemplar returns the asked-for target far below chance: it returns the other kind's target instead, so the operator does not merely help, it decides the answer. With no noise the read is exact; from noise 0.05 to 0.80 it holds between 0.927 and 0.990, and at 1.50 it breaks to 0.417 as both arms converge on chance.

The gate of §5.2 is sound here too. Its precision is 1.000 at all five kind counts, at coverage between 0.833 and 0.964. Taken through a Mantle lattice, where the

facets R	the score	cosine	dot product	ordered axis collapsed
1	1.000	1.000	1.000	0.438
2	1.000	1.000	0.000	0.000
3	1.000	0.975	0.000	0.000
4	1.000	0.5375	0.000	0.000
5	1.000	0.175	0.000	0.000
6	1.000	0.0625	0.000	0.000

cells	alignment alone	coverage	precision
where unguarded alignment is at chance (10)	0.075 or below	0.000 to 0.050	0.000 where it answers
with at least five answers to judge (15)	—	0.075 to 1.000	1.000

kinds R	chance	with the operator	operator removed	wrong-kind exemplar
2	0.500	0.882	0.500	0.118
3	0.333	0.912	0.333	0.056
4	0.250	0.944	0.250	0.021
6	0.167	0.981	0.167	0.002
8	0.125	0.979	0.125	0.007

store says which records are adjacent and nothing about how, the gated read scores 0.944, 0.917 and 0.972 at three, four and six kinds on one draw, against the in-memory gate's 0.889, 0.917 and 0.947 over twelve.

5.4 Counting relations up to the ceiling

Relation k owns a source direction u_k and a target direction w_k, and an edge of it puts one shared latent along u_k in its source's record and along w_k in its target's. The cross-covariance of the two endpoints then carries $u_k w_k^\top$, a matrix determined by which relation it is. Stacking every edge's flattened matrix gives the relational structure with nothing said about it, and the number of kinds is that stack's resolved rank. With $T = 96$, $D = 64$, 12 edges per relation and 8 draws, the count is exact in every draw at $R = 1$ through $R = 32$, at every noise level from 0.05 to 0.40. Edges separate by their kind with the number of kinds never supplied: within-kind alignment runs 0.903 to 0.958 against between-kind -0.129 to -0.953.

The largest count run at that width is its ceiling. Each relation owns two orthonormal directions, so R relations need $2R$ and the ceiling is $R = D/2$, which is 32 at 64 channels. Swept over 108 cells, four widths by three noise levels with R taken to each width's own ceiling, the count is exact at the ceiling at every width and every noise level: 8 at 16 channels, 16 at 32, 32 at 64 and 64 at 128, 4 of 4 ceilings. The generator refuses to build $R > D/2$ at every width rather than producing fewer distinct relations. The count is limited by how many independent role directions the space holds and by nothing else, and it is exact in all 108 cells, from one kind up to each ceiling.

The same counts come back through a Mantle lattice that holds one label for every edge and is never consulted for it: 2, 6, 8 and 10 kinds read directly and the same through the store, at $R = 2$, 4, 6 and 8.

5.5 Against a trained model, on seen and unseen entities

RESCAL [7] scores a triple with entity vectors and a matrix per relation, both fitted by gradient descent: the same object the score measures. It is written from scratch on the identical records, and every asymmetry favours it. It is given the relation index of every training triple, where the measured path gets no labels and takes its operator from an exemplar pair. Its latent dimension, learning rate and training budget are selected on a validation split, and the selected model fits its own training triples at 1.000 with validation top-1 at 1.000. Targets are drawn from a shared pool of 24, so each appears in many triples.

Where training is possible, training wins, by 0.095. Where there was no opportunity to train, the trained model is below chance and the measured path is unaffected, 0.875 against 0.172. That is not a deficiency of RESCAL: a method with per-entity parameters has no vector for an entity it never saw. The measured path scores 0.810 and 0.875 on the two blocks because nothing about it distinguishes a record it has seen from one it has not.

5.6 A relation's operator derived from its edges

A relation can hold no parameters at all and still have an operator: the mean of the matrices of the edges that cite it, taken at the moment it is asked for, so that an edge added or withdrawn changes the relation by entering or leaving the mean. The aggregate for a question is built only from edges that touch its source nowhere. On a lattice of 24 sources, more of the lattice seen is a better read at 8 of 8 noise levels. What improves is the ability to answer. At a relation holding a tenth of its strength, one edge answers 19.3% of questions at 0.828 accuracy, and 23 edges answer 94.0% at 1.000. Accuracy rises too,

	chance	trained RESCAL	measured path
seen entities: 42 held-out triples over trained sources	0.250	0.905	0.810
unseen entities: 64 questions over 16 sources in no training triple	0.250	0.172	0.875

but what the edges buy most is answers: the read stops abstaining.

Averaging trades noise for rank: the aggregate resolves 3.00 modes from one edge and 0.625 from 23. So the best backing is neither one edge nor all of them. On a relation that holds completely, six edges read 0.960 with a spread of 0.014 across five question seeds, against 0.668 with a spread of 0.045 for every edge, 6.2 spreads apart.

How many edges to use can be read off the relation itself: the longest run of its edges whose aggregate still resolves at the reader's own level. The rule reads only the relation, never the source, the candidates or the answer, so it asks one question. It is never worse than 0.950 over the noise range, where one edge falls to 0.167 and every edge to 0.667. Its cost is measured on a relation that is not one: it answers 0.147 of those questions against a single edge's 0.110, 0.25 spreads apart across six independently drawn lattices. A rule that retries the read until it resolves reads 5.69 spreads out on the same control and is refused. The control can only see an inflation of 0.297 or larger, so the admissible rule's cost is bounded there and not shown to be zero.

6. Where the method does not carry

6.1 The level the null delivers on autocorrelated rows

The permutation null treats T rows as T exchangeable observations. Measured on records independent by construction, each an autoregressive draw sharing nothing with any other over 125 rows and 16 channels, every pair must be refused, so the share called resolved is the realized false-alarm rate:

The caller asks for 0.05. From a row correlation of about 0.30 upward the null stops delivering it, and at 0.99 it delivers 0.757 on these 400 pairs and 0.748 on an independent draw of the same design: three pairs in four that share nothing are called related.

A phase-randomized surrogate that keeps each record's own covariance and spectrum exactly [8] holds the level at 0.065 or below across the range on these records. On a real corpus it removes the relation it is meant to be a null for: two records of the same activity have similar covariance because they are the same activity, and that survives the randomization. On a wide inertial corpus it takes the association on a same-activity task from 0.175 to 0.058, below a chance of 0.083.

What autocorrelated rows cost is the count. The variance of two series' cross-product runs over the product of their own autocorrelations [9], so the effective-sample factor is

$$\phi = 1 + 2\sum_{k} \left(1 - \tfrac{k}{T}\right) \rho_a(k)\, \rho_b(k), \qquad z' = z/\sqrt{\phi},$$

with ρ_a and ρ_b each record's own lag-k autocorrelation averaged over its channels. It is built from each record separately, so it cannot subtract a relation. It is inert where the defect is absent, a factor of 1.00 and a level of 0.045 at zero autocorrelation. It keeps its power, firing 1.000 on records sharing 0.6 or more of a trajectory at every autocorrelation, at a cost of 0.950 to 0.765 at row correlation 0.9 and the weakest sharing. Over four cut-offs from 10 to 125 lags on a 125-row axis the corrected level moves between 0.035 and 0.120 with no trend, so the taper does the damping and the cut-off does not matter. On the inertial corpus it leaves the read standing, 0.175 to 0.150, where a cosine reads 0.233.

A residual remains above a row correlation of 0.9, reaching 0.217 at 0.99, and is stated rather than closed. The generated corpora placed by their row correlation, the axis dial of §6.3 and the forward-pass corpus of §5.3, sit between -0.027 and $+0.011$, where the level delivered is 0.045, and at zero autocorrelation the correction's factor is 1.00, so it cannot move them. The correction is measured here and not applied inside the instrument.

6.2 The level at very few rows

The null's first two moments are exact. Its tail is the normal limit of the combinatorial central limit theorem [3], which is approximate when there are very few rows, and the decision inherits that. Refusal against how many observations of the same records have arrived, over 12 draws:

From 8 to 16 rows the decision refuses every question; from 24 rows upward it is sound and coverage rises with evidence. At four rows it answers a third of the questions and is wrong on a quarter of those, which is a defect. At four rows there are only $4! = 24$ permutations, so the exact tail can be enumerated rather than approximated; that repair changes no definition and adds no constant, and it belongs in the instrument.

6.3 Where the reads apply

The association measures co-variation along an ordered axis both records share. A record of a word in text is its context profile along position in the book, and word similarity does not vary along that axis. The condition is put to a controlled test, stated before the run: a dial sets how much of a relation lives on the axis and nothing else changes. Where none of it does, the association reads 0.115 against a chance of 0.125 while a vector cosine reads 0.390; with four tenths of it on the axis the

lag-one row correlation	0.00	0.30	0.60	0.80	0.90	0.95	0.99
as stated	0.045	0.065	0.190	0.312	0.515	0.647	0.757
with the effective-sample correction	0.045	0.050	0.058	0.065	0.080	0.102	0.217
the correction's factor	1.00	1.18	1.99	3.82	6.57	9.79	14.25

rows seen	4	8	12	16	24	32	48
answered	0.333	0.000	0.000	0.000	0.250	0.667	1.000
precision	0.750	—	—	—	1.000	1.000	1.000

association reads 0.980, from six tenths 1.000, and the cosine never passes 0.730. Measured on Moby-Dick and Alice, the 200 most similar held-out word pairs separate from median pairs by +0.113 and +0.121 on their profiles and by +0.069 and +0.004 on their trajectories along the axis: text sits at the end of the dial where the prediction is chance, and on a second-order task there the subspace read gives 0.297 and 0.333 against a vector cosine's 0.640 and 0.500. The reads apply to a corpus whose related records separate along the axis they share, and one pass over a corpus says whether it does.

6.4 Real corpora

The advantage of §5.1 does not reproduce on any real corpus measured.

The weather corpus is single-facet, where the one-facet control of §5.1 predicts a tie, and it ties. The air-quality corpus is admitted, multi-facet, and contains the narrow-loud configuration the claim is about, in 12 of 132 pairs; the two reads are not separated, at −0.86 paired standard errors. On the inertial corpus the association is behind by −2.32 paired standard errors, beaten by two of five scalar reads and ahead of the other three. Every method answers the same questions in each row, so each gap is scored on the questions the two disagree about.

The inertial corpus supplies the case the construction is built around. Each of its 900 questions is placed by the loudest distractor's peak channel agreement over the true match's, which exceeds 1 on 70.1% of them, with a median of 1.101. In five quantile bins the raw edge rises from −0.533 to −0.072, but both scores are falling toward chance there, so the share of the scalar's skill above chance that the association keeps is what survives a floor. It reads 0.42, 0.62, 0.53, 0.57 and 0.07. Where a magnitude ranking is most obstructed, at gains from 1.58 to 3.77, the association scores 0.089 against a chance of 0.083 while the scalar holds 0.161. The precondition is present in a fifth of the questions of a real corpus and the advantage is absent there.

6.5 Next-word prediction: the lookup wins

At word scale the operator reads context. Over 4000 held-out decisions with a vocabulary of 1024 it reaches a mean reciprocal rank of 0.1792 against 0.1365 for frequency alone, 6.3 standard errors. An n-gram lookup in its fit-free form, interpolating every matched order, reaches 0.2376, 8.6 standard errors ahead of the best operator. On the rarest quartile of continuations the operator reads 0.0114, level with a control that ranks by rarity and never consults the context at 0.0117, while the lookup reads 0.0430.

The strongest context read here weights every cached context by its match to the query and sums the votes, and the weighting decides whether the instrument carries it. With frames, cache, decisions, seed and candidates identical over 500 decisions, a cosine weighting reaches 0.2029, the association 0.1928 and the aperture 0.1868. The instrument's best is −0.52 standard errors against the cosine: no difference at this sample size, with the cosine nominally ahead. On this task the read that performs is a kernel regression, and the instrument is not load-bearing in it.

6.6 The reader on the store

The figures of §4 are the bare operator's, or one step of the reader. Five limits apply to the read of a store as a whole.

The headline is one store's. Resampling it, by dropping one statement in eight over twelve subsets, moves a reader step's 0.28 between 0.18 and 0.33 around a median of 0.28; dropping one in four moves it between 0.14 and 0.34. Subsets of one store are not independent of it, so this says how much of the figure is these particular statements, not what another ontology would give.

The relation has to be supplied. Told which relation acts, a reader step reaches the true destination on 28% of the 68 questions; choosing for itself, 13%. On the hand-written store choosing is level with a relation drawn at random, 13% against 9%, paired 6 to 3. On generated stores with part of a structure observed, choosing reads 11%, 12% and 12% at three shares observed, below a relation drawn at random at 38%, 40% and 44%, where being told reads 93%, 100% and 100%.

It has to be spelled as the store spells it. There is no vocabulary to match a near-miss against. The relation's identifier as written selects its operator on all 5 relations that have one; the word before its underscore, the word after, the two spaced and the two joined select it on none. A near-miss is not refused: the concept is still named, so the read runs with no relation selected, which is the choosing above.

corpus	admitted by the axis test	facets resolved	the association	the best scalar
two books (Moby-Dick; Alice)	no	—	0.297; 0.333	0.640; 0.500
weather stations, weekly	yes, 6.86 and 3.01	1.00 of 4	0.767	0.777
12 air-quality stations	yes, 66.76 and 10.99	1.75 of 11	0.497	0.511
912 inertial segments, 45 channels	no, 0.13 and −0.23	—	0.260	0.323

End to end the reader rarely says the answer. Each of the 84 statements held out in turn and asked back through the whole reader puts 68 questions; it says the removed answer on 2 of them, 3%, says something else on 17 and refuses 49, against a floor of 1.5%. Its floor is what costs most of that. Over 84 questions the reader reaches the answer 2.4% of the time as it stands and 24% with the floor removed, answering at all on 26% against 88%, and when it does answer it is right 9.1% of the time against 27%. The floor costs recall and precision both.

Inference past what the store states is shown on generated structure. Concepts are drawn from hidden groups, a relation maps group to group, and the store holds a random share of the true edges and nothing else. With 20% of the edges observed the read names a concept in the right group 94% of the time against a floor of 14%, rising to 100% at 80% observed against a floor of 5%, and every candidate is a pair the store is silent about. Over 20 further stores it runs between 82% and 98% around a median of 93%. These are class-level figures: the generated grader accepts more than one answer on 95% of its 319 questions, a median of 3, so the figure says the read reaches the right group and not the right member.

7. Where this sits in the literature

The association between two records being a matrix rather than a scalar is established: RESCAL [7], DistMult [10], R-GCN [11] and linearized attention [12] all carry a matrix-valued relation, TransE [5] carries a translation, tensor-product binding [13] binds by outer product, and a linear transformer is a fast-weight memory written by outer products [14]. Two things differ here.

The matrix is observed. RESCAL's $X_k \approx A R_k A^\top$ trains one R_k per relation. Here $M = \tilde{A}^\top \tilde{B}$ is a statistic of the statements, computed in one pass, with no target, no residual and nothing iterated. In every system above the matrix is obtained by gradient descent and the relation index arrives with the dataset.

The number of relations is counted. RESCAL's k ranges over a vocabulary that arrives with the dataset. Here an edge's kind is where its matrix sits, a position in a continuous space, and how many kinds a corpus contains is a rank, measured (§5.4). How many distinguishable kinds a given dimension can carry is bounded and proved (§9).

The attention it is measured against is the scaled dot product of [4], and the level it keeps is the false-alarm level of Neyman and Pearson [1]. The score's moments are the permutation moments of Pitman [2] and Hoeffding [3], the correction for autocorrelated rows follows Bartlett [9], and the phase-randomized surrogate is that of Prichard and Theiler [8].

8. Where the work stands, and what moves each row

Every row here is a position with a next step. A result measured under one set of choices is a fact about those choices, and the choice to vary is named.

The coordinate is independent of the order of the statements

A store is a set, so the order it was written down in carries nothing, and nothing derived from it moves when that order changes. The walk settles each vertex once, at the largest value that reaches it, which is a max-product shortest path. On the hand-written store, built in the given order and three others, the worst of 3809 non-zero coordinate cells moves 0, the read names 36.8% of 68 held-out statements on all four, and under three other hash seeds the read and the coordinate come back the same 3 times of 3.

The contrast keeps the same vertices and adds each one once per improving estimate, as its neighbours offer their settled values in the order the store lists them. That walk moves its worst cell 0.0795, the size of a contribution, and reads 35.3% on three orders and 36.8% on the fourth, because how many improvements a vertex sees is which neighbour the store happened to list first. A contribution per vertex is the only order-independent reading available: the other reading a decayed walk suggests, a vertex's contribution summed over every path, has the same total at every path length, because the step divides by the fanout and there are exactly that many neighbours.

Some questions are decided inside the arithmetic's own resolution

On the same 68, 3 are decided by a gap that the rounding bound of the dot product that produced it cannot resolve, the closest at 0 against a bound of 2.09e-14 at that score and that width, and all 3 are exact ties on the given order. The settled coordinate is the same on every order, so only the operator's rounding can move them. Scoring them either way is worth 4.4% of the figure, and how to score a question the arithmetic cannot separate is a decision the measurement surfaces.

The research coordinate and the shipped one agree to the bit

On the hand-written store, the prose spans and the two-scale prose store, a place settles 4.12 to 4.24 vertices on average before it reaches its aperture, and 0 of 97658917 cells differ between the research path and the shipped one, on the whole store and with a statement held out. The work is recorded as counts of vertices settled and neighbour entries offered, because those are the same on every machine and a wall clock is not.

Where the method carries is measured per store

§4.6 puts vehicle.n.01 at 2.9 times a draw among the reached, with one of twenty permuted stores reaching it, so that neighbourhood's figure does not clear the null. What predicts carrying is an open and tractable question: the leak counts of §3.3 report the properties that differ between the neighbourhoods that clear and those that do not, namely reciprocity, labels holding both directions and pairs under two labels, and the prediction is a measurement over more neighbourhoods.

Text carries differently in each representation

The spans tie counting at 7.0% of 114 held-out spans (§4.7), a fact about one representation at one scale; the four co-occurrence runs infer the relation from counts, a different object again. A store holding the sentence as a vertex as well lets two tokens in one sentence meet on an axis no span store produces, and against a null that keeps every sentence's size and every token's placements it reads 1.8%, below the null's 4.4%. On this body, grouping by sentence does not help the read; a body where sentences group tokens that belong together is the next measurement.

Scale is a growth exponent, and the aperture changes it

Read to its end the coordinate is dense at every size tried and grows at an exponent of 1.975; stopped at its own aperture it grows at 1.860 (§4.8), a change of growth and still not linear. The read gives a little for it on one of the three stores priced, 20.7% against 22.4%, and keeps or gains on the other two. The growth is measured from 50 to 705 statements, and the next range is the next measurement. The step past it is named: the hold-out removes one statement from a store and rebuilds the coordinate from nothing, where the settled walk's shortest-path tree can be repaired where the statement was.

A real working lattice

The read is not tested on a real working Mantle lattice, and that test needs a lattice built and verified for it.

The resolution of the nulls

Twenty draws is a floor of 0.048, and the nulls of §4.6 run at it. That is the finest any of them reports however the numbers fall, it is stated beside every share, and more draws give a finer one.

Ingestion, and why a curriculum has nothing to act through

The ingestion takes prose and returns statements with nothing in it an opinion: each position is cut at the extent its own middle earns and the extents written are reported. By default it also writes the sentence rung, the rung of the ladder that buys the most askable share for its read cost, and a caller can ask for the spans alone. Askable is not answered: on the body measured the sentence rung makes the read worse, 1.8% against 7.0% on the spans alone and below its own permutation null (§4.7), so the default buys reach and costs the read, and the spans alone are what the next-token figures are taken on. The ingestion writes the store the figures above are about, checked statement for statement: 2572 spans against 2572, 2242 placings against 2242, the same 120 sentences and every rung of the ladder identical. Every membership label carries a character the token reader cannot produce, and the collision count is taken: 0 at every rung.

A curriculum is an ordering of what a reader is shown. The operator is proved independent of the order statements arrive in (§9), the coordinate is independent of order (above), and the whole read path is measured under 4 arrival orders: the coordinate's worst cell moves 0 on the hand-written store and on the prose spans, the operator moves inside its rounding bound, and 1 of 68 answers on the hand-written store moves, a question decided inside the rounding, against 0 of 38 on prose. What a store answers depends on which statements are in it; the order they arrived in reaches only the one question decided inside the rounding.

A fitted model, and what to vary

On the hand-written store TransE at its best mean names 53.4% against 36.8%, and 61.6% against 40.7% among the reached (§4.9). Two things vary from here, each already built: the store's shape, which the prose stores change, and there TransE at its best names 3.1% against the derived read's 12.8% (§4.7); and the trivial arm of always naming the commonest destination, which is part of the comparison. On generated records the trained model wins where it has trained and falls below chance where it has not (§5.5).

9. Formal development (Lean)

The algebra the read rests on is formalized in Lean 4.31 against Mathlib, in seven files holding 42 theorems and 16 definitions. There is no `sorry`, no `admit` and no declared axiom; `#print axioms` on every headline theorem returns only `propext`, `Classical.choice` and `Quot.sound`, which are Lean's own.

theorem	what it establishes
carry_identity	$u(A^\top B) = (uA^\top)B$: the operator never has to be built (§2.5)
operator_append	a statement arriving adds one rank-one term and disturbs nothing already there (§2.4, §2.7)
operator_perm	the operator is independent of the order statements arrived in (§8)
alignment_sum_over_perms_eq_zero	the permutation null has mean zero on centred records, with nothing estimated (§2.6, §5)
sum_perm_alignment_sq	the null's second moment in closed form, $\mathrm{tr}(C_a C_b)/(T-1)$, so the null is exact with no sampling (§2.6, §5)
two_mul_le_finrank_of_roles	R relations own $2R$ orthonormal directions, so $2R \le \dim E$: the ceiling §5.4 measures being reached at every width
horizon_exists	reach is finite because signal decays past the floor, with no cap on hops; it supports the measured horizon of the ledger's propagation stage, which this paper does not carry
not_all_reads_are_stable	not every read is stable under the store gaining statements; stability is per read, and is checked

The last row is a negative result and is in the table on purpose. Three of these theorems carry the line of §5: the null's mean, its second moment, and the relation count, whose companion `roleMaps_linearIndependent` shows that with orthonormal target roles and no zero source role the R relation maps are linearly independent, so the span of the edges has dimension exactly R and a resolved rank recovers the count.

file	theorems	names
Carry.lean	5	carry_identity, operator_append, operator_append_sub, operator_perm, operator_nil
Arrival.lean	7	answer_congr, arrive_of_some, answer_arrive, not_all_reads_are_stable, answerable_arrive, arrival_keeps_every_answer, arrival_can_add_an_answerable_question
Permutation.lean	6	sum_perm_apply_const, sum_index_sum_perm, card_smul_sum_perm_apply, sum_perm_apply_eq_zero, alignment_factors_through_perm_sum, alignment_sum_over_perms_eq_zero
PairSum.lean	6	pairMover_fst, pairMover_snd, sum_perm_pair_const, offDiagonal_eq, sum_perm_offDiagonal, sum_perm_offDiagonal_centred
SecondMoment.lean	12	sum_perm_S_mul_eq, sum_perm_S_sq_eq, card_smul_P_diag, sum_offDiag_P, P_diag_mul, P_offDiag_const, sum_offDiag_count, P_offDiag_mul, sum_perm_S_sq, n_mul_P_gap, weighted_collapse, sum_perm_alignment_sq
RelationCount.lean	3	roleMap_apply, roleMaps_linearIndependent, two_mul_le_finrank_of_roles
Horizon.lean	3	below_floor_eventually, horizon_exists, constant_signal_never_falls

The second moment is stated division-free: on an axis of $n+1$ rows with both records centred, $n\sum_\pi S(\pi)^2 = |\mathrm{Perm}|\,(\sum a^2)(\sum b^2)$, and in the matrix case the right side is $|\mathrm{Perm}|\,\mathrm{tr}(C^A C^B)$. Divided through, these are the Pitman–Hoeffding moments [2, 3]. The route counts no permutations: the joint sum at two distinct indices is the same at every pair because the permutations act 2-transitively, and its total over all pairs is known in closed form. Both centring hypotheses do work. Centring a removes the off-diagonal product term, and centring b turns $\sum_{s\neq t} b_s$ into $-b_t$, which is where the $T-1$ comes from.

The proof is about the statistic the code computes

A theorem about a null is decoration if the implementation standardizes by something else, so the gap is closed from both ends in the package's tests. Every permutation is enumerated for a small axis, all 720 of them at six rows, and the closed form holds exactly rather than statistically. Both centring hypotheses are then broken deliberately, and the identity fails in each case. The instrument's standardized alignment is checked against the alignment divided by $\sqrt{(\sum a^2)(\sum b^2)/(n-1)}$ to 10^{-6} at eight, twelve and twenty-four rows, and in the matrix case against $\sqrt{\mathrm{tr}(C^A C^B)/(n-1)}$ to 10^{-6} at three, five and eight channels. The normalizer the code uses is the matrix functional, so the mechanism of §5.1 is a property of the code and not an interpretation of it.

What the theorems assume, and what is still owed

The counting argument is conditional on the roles being orthonormal, which is true of the generator by construction and is why the generator can say how many relations it made. It is not established for an arbitrary corpus, and nothing here claims it is: the theorems say what follows given that structure. That is the shape of the paper's central limit. The mechanism is proved and measured where the structure holds, and whether a real corpus has that structure is a separate question: §6.4 finds the advantage the structure produces absent on every real corpus

measured. The tail of the null is not proved exact; §6.2 records where its approximation shows.

Carried limits

Stated so they can be closed. The relation count is exact in every cell measured, up to the ceiling the basis sets. Refusal buys soundness with coverage, and the soundness holds where the decision answers at least five times: below that cut, three cells of the binding sweep answer, and every answer is wrong. What a relation's own choice of backing costs in false alarms is bounded at 0.297 and not shown to be zero. Above a row correlation of 0.9 the effective-sample correction leaves a residual, reaching 0.217 at 0.99. At four rows the decision answers on an approximate tail.

10. Code and reproduction

The package

The engine is the Python package `entroptics_infer`, built on the `entroptics` instrument [6]. The repository's README is how to use it. The methods of §2 map onto the `Engine` class:

§2	method
the basis (§2.2)	`Engine.basis()`
the walk and the place (§2.3)	`Engine.places()`, with `_settle`, `_resolved` and `_aperture`
the operator (§2.4)	`Engine.operator(r)`, from `bind.association_matrix`
the landing (§2.5)	`Engine.carry(a, r)`, never forming the operator
the score and the attention row (§2.6)	`Engine.attend(...)`; the attention layer is `DerivedAttention`, on `engine.associate`
a read under the protocol (§3.1)	`Engine.infer(a, r)`, holding out by default
learning (§2.7)	`Engine.relate(a, r, b)`
the leak counts (§3.3)	`entroptics_infer.survey`
ingestion (§4.7, §8)	`entroptics_infer.corpus(text)`, writing the sentence rung by default; `corpus(text, scales=())` writes the spans alone
the score, the refusal and the forward pass (§5)	`engine.associate`, `entroptics_infer.infer`, `entroptics_infer.relations`

No function in the package defaults the false-alarm level `far`. A read under the protocol:

```
a = engine.infer("dog", "is_a")
a.named      # 'mammal'
a.stated     # False: the store, as read, does not contain this
a.pool       # 65: candidates offered
a.reached    # 12: candidates this relation points into
elsewhere
```

The checks behind each result

Each result is a program under `research/checks/` or `research/experiments/` that prints its table, records what it measured under `research/results/`, and, for a check, decides its own verdict and exits non-zero on failure. Every figure is a row in `research/CLAIMS.md` naming the script that produced it.

result	check
§3.1 the coordinate leak	`research/checks/test_the_places_leak.py`, `research/checks/test_is_the_gap_real.py`, `research/checks/test_what_the_bare_operator_carries_on_the_ontology.py`
§3.3 the coordinate leak	`research/checks/test_the_places_leak.py`
§4.1 the attention layer	`research/checks/test_whether_the_torch_layer_has_anything_to_train.py`
§4.2 the bare operator	`research/checks/test_what_the_bare_operator_carries_on_the_ontology.py`
§4.3 the generated store	`research/checks/test_what_the_bare_operator_carries.py`
§4.4 one implementation	`research/checks/test_whether_the_shipped_engine_reads_what_the_research_one_reads.py`
§4.5 a relation arriving	`research/checks/test_whether_a_fact_arrives_without_retraining.py`
§4.6 the controls	`research/checks/test_whether_the_read_survives_its_controls.py`
§4.7 the co-occurrence runs	the ledger rows listed below
§4.7 next token on spans	`research/checks/test_whether_the_read_predicts_the_next_token.py`
§4.7 the shape of prose	`research/checks/test_what_shape_prose_has_to_be_ingested_in.py`
§4.7 the ordering	`research/checks/test_where_the_right_token_sits_when_the_read_misses.py`
§4.7 crowded relations	`research/checks/test_whether_the_read_fails_where_the_relation_is_crowded.py`
§4.7 TransE on prose	`research/checks/test_what_a_trained_model_gets_on_prose.py`
§4.7 the spans the ingestion writes	`research/checks/test_whether_the_shipped_ingestion_writes_the_measured_store.py`
§4.7 the scales of prose	`research/checks/test_what_each_scale_of_prose_buys.py`
§4.7 the sentence rung	`research/checks/test_whether_prose_at_two_scales_changes_the_read.py`
§4.8 growth read to its end	`research/checks/test_whether_truncation_changes_the_growth.py`
§4.8 the aperture	`research/checks/test_whether_the_walk_can_stop_at_its_own_aperture.py`
§4.8 the cost regimes	`research/checks/test_what_a_read_costs_as_the_store_grows.py`
§4.9 TransE on the same store	`research/checks/test_what_a_trained_model_gets_on_the_same_store.py`
§4.9 TransE on prose	`research/checks/test_what_a_trained_model_gets_on_prose.py`
§5.1 selection	`research/experiments/exp01_association_vs_scalar.py`, record `research/results/exp01.json`
§5.1 the held width and noise swept	`research/experiments/exp20_audit_selection.py`, record `research/results/exp20.json`

result	check
§5.2 refusal	research/experiments/exp02_refusal.py, record research/results/exp02.json
§5.3 the forward pass	research/experiments/exp05_forward_pass.py, record research/results/exp05.json
§5.4 the relation count	research/experiments/exp03_relation_types.py, record research/results/exp03.json
§5.4 the count to its ceiling	research/experiments/exp23_audit_relation_count.py, record research/results/exp23.json
§5.5 RESCAL	research/experiments/exp10_trained_baseline.py, record research/results/exp10.json
§5.6 the relation's backing	research/experiments/exp42_relation_vertex.py, record research/results/exp42.json
§5.6 the admissible backing rule	research/experiments/exp44_backing_from_the_relation.py, record research/results/exp44.json
§6.1 the level on autocorrelated rows	research/experiments/exp45_null_on_autocorrelated_rows.py, record research/results/exp45.json
§6.1 the surrogate on the wide corpus	research/experiments/exp46_wide_corpus_corrected_null.py, record research/results/exp46.json
§6.1 the effective-sample correction	research/experiments/exp47_effective_sample_size.py, record research/results/exp47.json
§6.2 the level at few rows	research/experiments/exp06_learning.py, record research/results/exp06.json
§6.3 the axis dial	research/experiments/exp09_axis_condition.py, record research/results/exp09.json
§6.3 where text sits	research/experiments/exp11_text_diagnostics.py, record research/results/exp11.json
§6.4 the books	record research/results/exp11.json
§6.4 weather	research/experiments/exp13_weather.py, record research/results/exp13.json
§6.4 air quality	research/experiments/exp14_multichannel.py, record research/results/exp14.json
§6.4 the inertial corpus	research/experiments/exp19_wide_corpus.py, record research/results/exp19.json
§6.4 the crossover strata	research/experiments/exp25_crossover_strata.py, record research/results/exp25.json
§6.5 the lookup	research/experiments/exp31_rank_and_rarity.py, record research/results/exp31.json
§6.5 the weighting	research/experiments/exp35_weighting_read.py, record research/results/exp35.json
§6.6 the one store	research/checks/test_how_much_the_store_decides.py
§6.6 told or choosing	research/checks/test_naming_the_relation.py, research/checks/test_whether_choosing_beats_no_rule_on_the_real_store.py, research/checks/test_choosing_the_relation_to_infer_with.py
§6.6 the spelling	research/checks/test_how_a_question_can_name_a_relation.py
§6.6 end to end	research/checks/test_what_the_reader_does_today.py, research/checks/test_what_the_reader_does_with_no_floor.py
§6.6 a partly observed structure	research/checks/test_inference_on_a_partly_stated_structure.py, research/checks/test_how_much_the_generator_decides.py, research/checks/test_what_the_generated_grader_can_ask.py
§8 the controls	research/checks/test_whether_the_read_survives_its_controls.py
§8 the shape of prose	research/checks/test_what_shape_prose_has_to_be_ingested_in.py
§8 next token on spans	research/checks/test_whether_the_read_predicts_the_next_token.py
§8 growth read to its end	research/checks/test_whether_truncation_changes_the_growth.py
§8 the aperture	research/checks/test_whether_the_walk_can_stop_at_its_own_aperture.py
§8 TransE on the same store	research/checks/test_what_a_trained_model_gets_on_the_same_store.py
§8 TransE on prose	research/checks/test_what_a_trained_model_gets_on_prose.py
§8 order	research/checks/test_whether_the_order_a_store_was_given_in_reaches_the_read.py
§8 agreement to the bit	research/checks/test_whether_the_same_coordinate_can_be_built_faster.py
§8 ingestion	research/checks/test_whether_the_shipped_ingestion_writes_the_measured_store.py
§8 arrival orders	research/checks/test_whether_a_curriculum_can_reach_a_read.py
§8 the sentence rung	research/checks/test_whether_prose_at_two_scales_changes_the_read.py
§8 RESCAL	record research/results/exp10.json
§9 the second moment, enumerated	tests/test_second_moment.py

Each result above is backed by rows of the ledger, research/CLAIMS.md:

result	CLAIMS rows
the coordinate leak	CLAIMS §47.1, §47.3, §47.153
the bare operator	CLAIMS §47.153
the generated store	CLAIMS §47.154
the controls	CLAIMS §47.155
the co-occurrence runs	CLAIMS §7.2, §7.2b, §7.2c, §7.3, §7.11
the shape of prose	CLAIMS §47.157
the ordering	CLAIMS §47.174
crowded relations	CLAIMS §47.175
the spans the ingestion writes	CLAIMS §47.171
the scales of prose	CLAIMS §47.170
growth read to its end	CLAIMS §47.156
the aperture	CLAIMS §47.169
TransE on the same store	CLAIMS §47.159
selection	CLAIMS §1.5–§1.9

result	CLAIMS rows
the held width and noise swept	CLAIMS §17.1–§17.4
refusal	CLAIMS §2.1–§2.3
the forward pass	CLAIMS §5.1–§5.6
the relation count	CLAIMS §3.1–§3.7
the count to its ceiling	CLAIMS §20.1–§20.5
RESCAL	CLAIMS §9.1–§9.5
the relation's backing	CLAIMS §31.2–§31.9
the admissible backing rule	CLAIMS §33.2–§33.5
the level on autocorrelated rows	CLAIMS §34.1–§34.7
the surrogate on the wide corpus	CLAIMS §35.1
the effective-sample correction	CLAIMS §36.4–§36.9
the level at few rows	CLAIMS §6.4
the axis dial	CLAIMS §7.14
where text sits	CLAIMS §7.15, §7.11
the books	CLAIMS §7.11
weather	CLAIMS §11.1–§11.5
air quality	CLAIMS §12.1–§12.5
the inertial corpus	CLAIMS §16.3
the crossover strata	CLAIMS §22.1–§22.4
the lookup	CLAIMS §25.1–§25.4
the weighting	CLAIMS §28.1–§28.3
the one store	CLAIMS §47.25
told or choosing	CLAIMS §47.7, §47.111, §47.72
the spelling	CLAIMS §47.77, §47.78
end to end	CLAIMS §47.84, §47.131, §47.133
a partly observed structure	CLAIMS §47.48, §47.79, §47.88
order	CLAIMS §47.162
agreement to the bit	CLAIMS §47.163
the second moment, enumerated	CLAIMS §10.14, §10.19

The checks that read a real Mantle lattice, named by `ENTROPTICS_LATTICE`, skip without one, exit 0 and write no record. The generated records of §5 and §6 are built at the seed each script states and none is read from disk; the real corpora are fetched by the script that uses them, which records the SHA-256 of what it received.

Re-running, and the gates

```
for f in research/checks/test_*.py; do python "$f"; done  # the
55 research checks, in turn
python research/verify.py                                  #
re-run the experiments against the register
python research/verify.py --paths                         # the
register against the stored results
python research/verify_records.py                          #
every record against a fresh run of its check
python research/checks/test_the_ledger_is_consistent.py    #
the ledger's addresses and citations
python research/prose_check.py                             #
every figure quoted against its source
pytest                                                     # the
package's own suite
```

The loop runs every one of the 55 files in `research/checks/`; none is excluded. `verify_records.py` re-runs every check that has a record and fails on any figure that moved; `prose_check.py` checks every figure quoted in this paper against the records its section maps to in the table above; the ledger gate checks that every address in `research/CLAIMS.md` is unique, every citation resolves and every script a row names exists.

The formal development lives in `research/lean/` and builds standalone: `lake exe cache get && lake build` against the Mathlib pinned in `research/lean/lakefile.toml`, and `lake env lean axioms.lean` prints the axiom listing. The theorem count is recorded in `research/results/lean.json`. The tests enumerating every permutation against the closed form, and checking the instrument against it, are `tests/test_second_moment.py`.

References

[1] J. Neyman and E. S. Pearson. On the problem of the most efficient tests of statistical hypotheses. Philosophical Transactions of the Royal Society of London, Series A, 231:289–337, 1933.

[2] E. J. G. Pitman. Significance tests which may be applied to samples from any populations. II. The correlation coefficient test. Supplement to the Journal of the Royal Statistical Society, 4(2):225–232, 1937.

[3] W. Hoeffding. A combinatorial central limit theorem. Annals of Mathematical Statistics, 22(4):558–566, 1951.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin. Attention is all you need. NeurIPS 2017.

[5] A. Bordes, N. Usunier, A. García-Durán, J. Weston and O. Yakhnenko. Translating embeddings for modeling multi-relational data. NeurIPS 2013.

[6] Sessford, I. J. Entroptics: reading a 2-D signal as a finite optical aperture at its own entropy-matched resolution. Pre-print, 2026. Software version 0.2.12, version DOI 10.5281/zenodo.24114784; the concept DOI 10.5281/zenodo.21273400 resolves to the latest release. https://github.com/Agience/entroptics

[7] M. Nickel, V. Tresp and H.-P. Kriegel. A three-way model for collective learning on multi-relational data. ICML 2011.

[8] D. Prichard and J. Theiler. Generating surrogate data for time series with several simultaneously measured variables. Physical Review Letters, 73(7):951–954, 1994.

[9] M. S. Bartlett. Some aspects of the time-correlation problem in regard to tests of significance. Journal of the Royal Statistical Society, 98(3):536–543, 1935.

[10] B. Yang, W. Yih, X. He, J. Gao and L. Deng. Embedding entities and relations for learning and inference in knowledge bases. ICLR 2015.

[11] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov and M. Welling. Modeling relational data with graph convolutional networks. ESWC 2018.

[12] A. Katharopoulos, A. Vyas, N. Pappas and F. Fleuret. Transformers are RNNs: fast autoregressive transformers with linear attention. ICML 2020.

[13] P. Smolensky. Tensor product variable binding and the representation of symbolic structures in connectionist systems. Artificial Intelligence, 46(1–2):159–216, 1990.

[14] I. Schlag, K. Irie and J. Schmidhuber. Linear transformers are secretly fast weight programmers. ICML 2021.